

# 4CS02 – Cryptologie

## Travaux Dirigés

### Cryptographie à clé secrète

Année 2025

## 1 Schéma de Feistel

### 1.1 Schéma de Feistel à trois tours

Un réseau ou chiffrement de Feistel est subdivisé en plusieurs étages (tours). Le message est découpé en blocs qui subissent une suite de transformations. Un chiffrement de Feistel est un chiffrement itéré qui utilise à chaque tour le même schéma (dit schéma de Feistel : cf. Figure 1 avec  $L$  pour Left et  $R$  pour Right) pour chiffrer un bloc.

Ce type de construction est à la base de l'algorithme de chiffrement DES (Data Encryption Standard) que nous avons étudié en cours, et qui est un schéma de Feistel à 16 tours. Nous considérons dans cet exercice une version “simplifiée” du DES : un schéma de Feistel à trois tours. Les trois fonctions  $f_1$ ,  $f_2$  et  $f_3$  constituent ici la clé de chiffrement (les fonctions  $f_i$  sont appelées fonctions de confusion) et sont définies dans la Table 1.

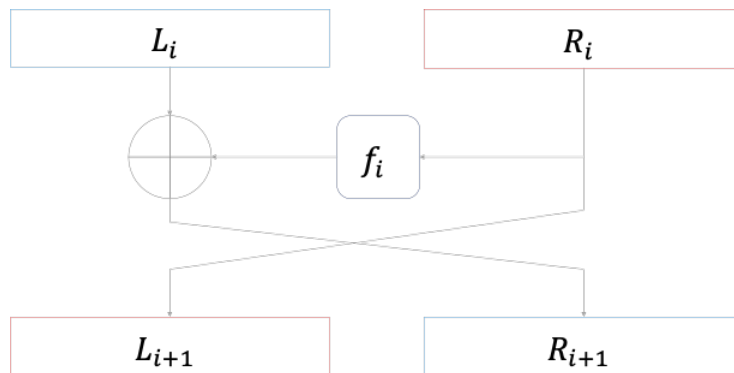


FIGURE 1 – Schéma de Feistel

|         |     |   |     |         |     |   |     |         |     |   |     |
|---------|-----|---|-----|---------|-----|---|-----|---------|-----|---|-----|
| $f_1 :$ | 000 | → | 110 | $f_2 :$ | 000 | → | 010 | $f_3 :$ | 000 | → | 111 |
|         | 001 | → | 100 |         | 001 | → | 011 |         | 001 | → | 010 |
|         | 010 | → | 111 |         | 010 | → | 110 |         | 010 | → | 110 |
|         | 011 | → | 000 |         | 011 | → | 111 |         | 011 | → | 110 |
|         | 100 | → | 110 |         | 100 | → | 000 |         | 100 | → | 000 |
|         | 101 | → | 010 |         | 101 | → | 101 |         | 101 | → | 101 |
|         | 110 | → | 001 |         | 110 | → | 110 |         | 110 | → | 100 |
|         | 111 | → | 101 |         | 111 | → | 110 |         | 111 | → | 001 |

TABLE 1 – Fonctions  $f_i$

Avec les définitions suivantes pour le chiffrement, pour  $i \in [1, 3]$  :

$$L_{i+1} = R_i, R_{i+1} = L_i \oplus f_i(R_i).$$

Au final, nous obtenons le message chiffré  $L_4 \| R_4$  pour le message clair  $L_1 \| R_1$

1. Ecrire les étapes de la fonction de déchiffrement qui associe à un message chiffré  $L_4 \| R_4$  son message en clair  $L_1 \| R_1$
2. Considérons le message à six bits 101110. Le message est découpé en deux parties de trois bits  $L_1 = 101$  et  $R_1 = 110$ . Quel est le message chiffré après les trois tours de Feistel ?

## 1.2 Faiblesse du schéma de Feistel à deux tours

Nous nous plaçons dans le cas d'un attaquant capable de demander des chiffrés  $C$  sur des messages  $M$  de son choix. L'attaquant sait que c'est un schéma de Feistel à deux tours qui est utilisé, mais il ne connaît pas la clé secrète utilisée. Montrer comment un tel attaquant peut distinguer ce schéma de Feistel à deux tours d'une fonction aléatoire. Plus précisément, chaque fois que l'attaquant envoie un message  $M$  de son choix, il obtient soit le chiffré de  $M$ , soit une valeur aléatoire. Est-il capable de faire la différence ?

## 2 Mode CTR et XOR-malléabilité

Nous considérons le schéma de chiffrement AES utilisé avec le mode CTR, l'ensemble donnant un algorithme que nous noterons **SymEnc**. Pour cet exercice, pour tout  $x$ , nous notons  $\langle x \rangle$  le nombre  $x$  en développement binaire.

Pour rappel, le chiffrement en mode CTR fonctionne de la façon suivante pour un message  $P$  séparé en  $n$  blocs  $(P_1, \dots, P_n)$  de 128 bits chacun :

- générer  $r \in \{0, 1\}^{128}$ . Les valeurs successives (les *counter*)  $\langle r \rangle, \langle r+1 \rangle, \langle r+2 \rangle, \dots$  sont définis en utilisant l'addition dans  $\mathbb{Z}/2^{128}\mathbb{Z}$  ;
- calculer le chiffré

$$c = \text{SymEnc}(k, P) = [r, \text{AES}(k, \langle r+1 \rangle) \oplus P_1, \dots, \text{AES}(k, \langle r+n \rangle) \oplus P_n].$$

Soit un chiffré  $C = [r, c]$  qui a été généré comme  $\text{SymEnc}(k, P)$  avec  $P$  un message clair de taille 128 bits. Soit  $P'$  un message différent de  $P$ . Vous êtes l'adversaire, vous connaissez seulement  $[P, C, P']$ .

Calculer le chiffré  $C' = [r', c']$ , constitué de 2 blocs de 128 bits, tel que le déchiffrement donne  $P'$ .

### 3 Exécution d'AES

La version de Rijndael standardisée par le NIST, appelée *chiffrement AES*, est un chiffrement par bloc de 128 bits de type réseau de substitutions-permutations. Les clés peuvent être de 128, 192 ou 256 bits. Le chiffrement découpe chaque bloc de 128 bits en 16 octets et chaque bloc est représenté sous forme d'une matrice carrée 4x4 où les octets sont numérotés de 1 à 16 de gauche à droite et de haut en bas.

Ce bloc est initialement le bloc de message clair à chiffrer auquel on additionne bit à bit une clé de tour initial  $K_0$ . Le chiffrement AES applique ensuite de façon répétée quatre procédures appelées **SubBytes**, **ShiftRows**, **MixColumns** et **AddRoundKey** sur la matrice carrée 4 X 4 appelée *matrice état*.

- La procédure « substitution des octets » (**SubBytes**) applique sur chacun des seize octets de l'état interne une S-boîte, assure une bonne confusion et a pour but de faire disparaître les structures tant linéaires qu'algébriques du chiffrement. Si la fonction qui permet de passer d'un octet à un autre peut paraître un peu complexe (voir le cours), l'implémentation utilise généralement une table de correspondance, comme nous allons le voir ci-dessous.
- La procédure « décalage de lignes » (**ShiftRows**) effectue des rotations vers la gauche sur les lignes de la matrice état. Le décalage pour chaque ligne est différent et s'effectue de la façon suivante : (i) la première ligne n'est pas modifiée, (ii) la seconde ligne est décalée d'une case vers la gauche, (iii) la troisième ligne est décalée de deux cases et (iv) la dernière ligne est décalée de trois cases.
- La procédure « mélange des colonnes » (**MixColumns**) combine les 4 octets de chaque colonne de la matrice état en appliquant une transformation linéaire. Le but de ces deux dernières opérations est d'assurer une bonne diffusion de l'information.
- La procédure « RoundKey » il s'agit d'un XOR avec la clé du tour

#### 3.1 Chiffrement AES

Nous allons maintenant exécuter un tour complet d'AES. Pour cela, nous considérons le texte suivant à chiffrer :

|    | x0 | x1 | x2 | x3 | x4 | x5 | x6 | x7 | x8 | x9 | xa | xb | xc | xd | xe | xf |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0x | 63 | 7c | 77 | 7b | f2 | 6b | 6f | c5 | 30 | 01 | 67 | 2b | fe | d7 | ab | 76 |
| 1x | ca | 82 | c9 | 7d | fa | 59 | 47 | f0 | ad | d4 | a2 | af | 9c | a4 | 72 | c0 |
| 2x | b7 | fd | 93 | 26 | 36 | 3f | f7 | cc | 34 | a5 | e5 | f1 | 71 | d8 | 31 | 15 |
| 3x | 04 | c7 | 23 | c3 | 18 | 96 | 05 | 9a | 07 | 12 | 80 | e2 | eb | 27 | b2 | 75 |
| 4x | 09 | 83 | 2c | 1a | 1b | 6e | 5a | a0 | 52 | 3b | d6 | b3 | 29 | e3 | 2f | 84 |
| 5x | 53 | d1 | 00 | ed | 20 | fc | b1 | 5b | 6a | cb | be | 39 | 4a | 4c | 58 | cf |
| 6x | d0 | ef | aa | fb | 43 | 4d | 33 | 85 | 45 | f9 | 02 | 7f | 50 | 3c | 9f | a8 |
| 7x | 51 | a3 | 40 | 8f | 92 | 9d | 38 | f5 | bc | b6 | da | 21 | 10 | ff | f3 | d2 |
| 8x | cd | 0c | 13 | ec | 5f | 97 | 44 | 17 | c4 | a7 | 7e | 3d | 64 | 5d | 19 | 73 |
| 9x | 60 | 81 | 4f | dc | 22 | 2a | 90 | 88 | 46 | ee | b8 | 14 | de | 5e | 0b | db |
| ax | e0 | 32 | 3a | 0a | 49 | 06 | 24 | 5c | c2 | d3 | ac | 62 | 91 | 95 | e4 | 79 |
| bx | e7 | c8 | 37 | 6d | 8d | d5 | 4e | a9 | 6c | 56 | f4 | eA | 65 | 7a | ae | 08 |
| cx | ba | 78 | 25 | 2e | 1c | a6 | b4 | c6 | e8 | dd | 74 | 1f | 4b | bd | 8b | 8a |
| dx | 70 | 3e | b5 | 66 | 48 | 03 | f6 | 0e | 61 | 35 | 57 | b9 | 86 | c1 | 1d | 9e |
| ex | e1 | f8 | 98 | 11 | 69 | d9 | 8e | 94 | 9b | 1e | 87 | e9 | ce | 55 | 28 | df |
| fx | 8c | a1 | 89 | 0d | bf | e6 | 42 | 68 | 41 | 99 | 2d | 0f | b0 | 54 | bb | 16 |

TABLE 2 – S-Box AES

$$M = \begin{bmatrix} 00 & 16 & 1a & 17 \\ 04 & 1c & 00 & 07 \\ 17 & 0e & 03 & 01 \\ 1b & 0f & 0f & 10 \end{bmatrix}$$

1. Tout d'abord, en utilisant la S-Box de la Table 2, appliquer l'opération **SubBytes** sur les message ci-dessus.
2. Appliquer l'opération **ShiftRows** en appliquant les décalages décrits ci-dessus.
3. Appliquer l'opération **MixColumns** afin d'obtenir la première case de l'état. Il s'agit d'un produit matriciel avec la matrice

$$C = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

Pour effectuer une telle opération, il faut de travailler dans le corps  $GF(2^8) = F_2[x]/P$  avec  $P(x) = x^8 + x^4 + x^3 + x + 1$ , c'est-à-dire que chaque octet va être représenté par un élément de  $GF(2^8)$  et chaque opération sur les polynômes sera réduite modulo le polynôme  $P(x) = x^8 + x^4 + x^3 + x + 1$ . Par exemple, l'octet

$$\alpha = 54 = 01010100 = x^6 + x^4 + x^2.$$

|        |    |    |    |    |    |    |    |    |    |    |
|--------|----|----|----|----|----|----|----|----|----|----|
| $i$    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| $rc_i$ | 01 | 02 | 04 | 08 | 10 | 20 | 40 | 80 | 1b | 36 |

TABLE 3 – Table des valeurs de  $rc_i$

Vous pourrez continuer de calculer le reste de la matrice, soit à la main (!), soit en utilisant un ordinateur. Dans ce dernier cas, vous pouvez trouver un code existant pour une telle opération mathématique, ou bien créer le vôtre.

4. Appliquer enfin le **AddRoundKey** en prenant comme clé du premier tour :

$$K = \begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix}$$

### 3.2 Diversification de clé de l’AES

Dans cet exercice, nous considérons la clé initiale

$$K_0 = \begin{bmatrix} 00 & 11 & 22 & 33 \\ 44 & 55 & 66 & 77 \\ 88 & 99 & AA & BB \\ CC & DD & EE & FF \end{bmatrix}$$

La clé est de longueur 128 bits, présentée dans une matrice  $4 \times 4$  (on parle de 4 mots de 4 octets en lisant par colonne). L’objectif de cette étape est d’étendre cette clé pour qu’une clé de 4 mots différents soit utilisée à chaque tour. Nous avons donc besoin de  $4 \cdot (10 + 1) = 44$  mots de 4 octets (10 tours plus 1 **AddRoundKey** initial). Pour cela, nous allons partir de la clé  $K_0$  ci-dessus et en exécutant le processus de diversification de clé de l’AES défini par la fonction suivante :

$$W_i = \begin{cases} K_i & \text{si } i < 4 \\ W_{i-4} \oplus \text{ByteSub}(\text{RotWord}(W_{i-1})) \oplus rcon_{i/4} & \text{si } i \geq 4 \text{ et } i \equiv 0 \pmod{4} \\ W_{i-4} \oplus W_{i-1} & \text{sinon} \end{cases}$$

La fonction **ByteSub** est la fonction de substitution d’AES défini plus haut à la Figure 2, **RotWord** est une rotation circulaire d’un octet et  $rcon_i = [rc_i \ 00 \ 00 \ 00]$  est un mot de 4 octets avec  $rc_i$  une constante définie par la Table 3.

Donner les 8 premiers mots de 4 octets.

## 4 Analyse de Camellia

Dans le protocole IKEv2, notamment utilisé dans les télécommunications, l’un des algorithmes proposé correspond à Camellia en mode CCM. La structure de Camellia est fournie en Figure 2, où les clés  $k_i$ ,  $k_l$  et  $kw_j$  sont dérivées

de la clé  $k_s$ .

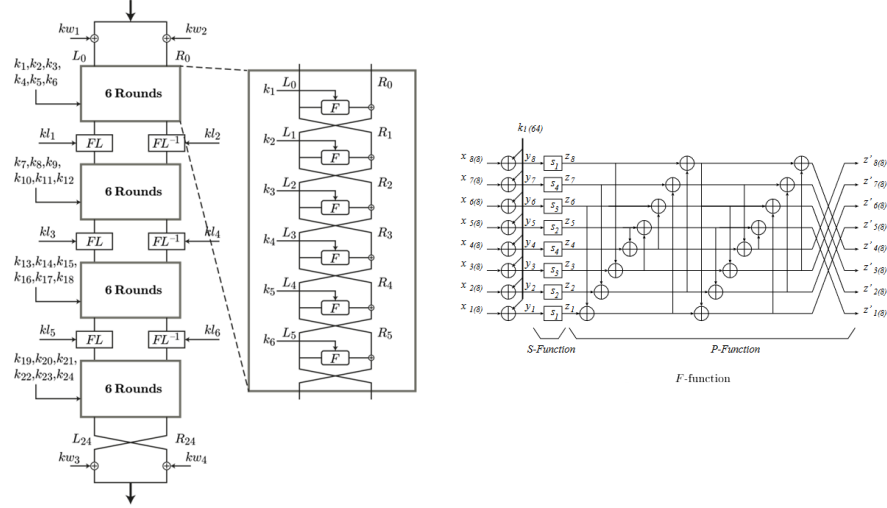


FIGURE 2 – Structure de Camellia

1. Quelle(s) structure(s) connue(s) reconnaissez-vous dans le schéma Camellia ? A quel(s) endroit(s) ?
2. Expliquez avec un maximum de détails en quoi vous retrouvez dans Camellia les principes fondamentaux introduits par Shannon pour les systèmes de chiffrement par blocs.
3. Le mode CCM (counter with cipher block chaining message authentication code) combine le mode CTR avec le mode CBC, utilisés en version “MAC-then-Encrypt”. Faites le diagramme d’un tel mode.

## 5 Fonction de substitution d’AES

Dans cet exercice, nous allons étudier la fonction de substitution d’AES. Pour rappel, il s’agit d’une fonction fixe de 8 bits vers 8 bits, exécutée sur chaque octet de l’état.

Cette opération se base sur une opération algébrique dans  $GF(2^8) = F_2[X]/P$  où  $P(X) = X^8 + X^4 + X^3 + X + 1$ . Ainsi, chaque octet va être représenté par un élément de  $GF(2^8)$  (réduits modulo  $P$ ; de degré inférieur à 8). Par exemple, l’octet 54 = 01010100 sera représenté par  $X^6 + X^4 + X^2$ . Pour rappel, les coefficients sont toujours dans  $\{0, 1\}$ .

La fonction ByteSub va s’effectuer en deux étapes :

1. Calculer l’inverse de l’octet  $a$  dans  $GF(2^8) = F_2[X]/P$ .

2. Calculer l'image du résultat précédent par la fonction  $G : GF(2^8) \rightarrow GF(2^8)$  telle que

$$G(X) = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \cdot X \oplus \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \text{ où } X = \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{pmatrix}$$

où chaque  $x_i \in \{0, 1\}$ .

Nous allons considérer l'octet  $\alpha = 1f$ .

1. Représenter cet octet  $\alpha$  par son polynôme  $A$  correspondant  $GF(2^8) = F_2[X]/P$  où  $P(X) = X^8 + X^4 + X^3 + X + 1$ .
2. Exécuter l'algorithme d'Euclide étendu sur les polynômes  $P$  et  $A$ , afin de trouver les polynômes  $U$  et  $V$  tel que  $U \cdot P + V \cdot A = 1$ .
3. En déduire l'inverse  $A^{-1}$  du polynôme  $A$  modulo  $P$ . Finalement, en déduire l'inverse  $\alpha^{-1}$  de l'octet  $\alpha$  dans  $GF(2^8) = F_2[X]/P$ .
4. Calculer l'image de  $\alpha^{-1}$  par la fonction  $G$  ci-dessus. Vérifier que la sortie obtenue correspond bien à celle attendue dans la table de correspondance du ByteSub d'AES vu en cours.